

SCHOOL OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Course Project Assignment

Machine Learning & Deep Learning

Total Points:	60 Points
Team Size:	Solo or Pairs (max 2 students)
Deliverable:	Code + Written Report + Demo
Submission:	Via course portal
Academic Year:	2024–2025

Topics Covered

1. Classification Algorithms
2. Python Implementation of Classification Models
3. Dimensionality Reduction Techniques
4. Deep Learning Fundamentals
5. Convolutional Neural Networks (CNNs)
6. General Overview of Deep Learning Algorithms

Contents

1	Project Overview	3
1.1	Learning Objectives	3
1.2	Grade Distribution Summary	3
2	Team Policy	3
3	Task 1 — Classification Algorithms	5
3.1	Overview	5
3.2	Required Algorithms	5
3.3	Deliverables	5
3.4	Useful Resources	5
4	Task 2 — Python Implementation of Classification Models	6
4.1	Overview	6
4.2	Dataset Requirements	6
4.3	Required Pipeline	6
4.4	Code Snippet Guidance	6
5	Task 3 — Dimensionality Reduction Techniques	8
5.1	Overview	8
5.2	Techniques to Cover	8
5.3	Deliverables	8
6	Task 4 — Deep Learning Fundamentals	9
6.1	Overview	9
6.2	Core Concepts	9
6.3	Deliverables	9
7	Task 5 — Convolutional Neural Networks	11
7.1	Overview	11
7.2	Dataset	11
7.3	Deliverables	11
8	Task 6 — General Overview & Integration	13
8.1	Overview	13
8.2	Deliverables	13
9	Submission Guidelines	14
9.1	Required Files	14
9.2	Report Format	14
9.3	Evaluation Rubric	14
10	Recommended Resources	15
10.1	Books	15
10.2	Online Courses & Tutorials	15
10.3	Key Python Libraries	15
10.4	Datasets	15

11 Frequently Asked Questions	16
12 Grading Checklist	17

1. Project Overview

Project Description

This project challenges you to integrate theoretical knowledge and practical skills across the core pillars of modern Machine Learning and Deep Learning. You will design, implement, evaluate, and critically analyze a complete ML/DL pipeline — demonstrating mastery of classical algorithms, neural architectures, and best software practices.

The project is divided into **six interconnected tasks**, each targeting a specific topic from the course. Tasks are designed to build progressively upon each other, culminating in a comprehensive report and code submission. Students may work **individually or in pairs**.

1.1 Learning Objectives

Upon successful completion of this project, students will be able to implement and compare multiple classification algorithms from scratch and using libraries, apply dimensionality reduction techniques and interpret their effect on model performance, and design and train deep learning models using modern frameworks. Additionally, students will build, tune, and evaluate Convolutional Neural Networks on image data, demonstrate critical understanding of algorithm trade-offs, limitations, and use cases, and produce professional-quality technical documentation and reproducible code.

1.2 Grade Distribution Summary

Task	Title	Points	%
Task 1	Classification Algorithms	10	16.7%
Task 2	Python Implementation of Classification Models	10	16.7%
Task 3	Dimensionality Reduction Techniques	10	16.7%
Task 4	Deep Learning Fundamentals	10	16.7%
Task 5	Convolutional Neural Networks	10	16.7%
Task 6	General Overview & Integration	10	16.7%
Total		60	100%

2. Team Policy

Collaboration Guidelines

Projects may be completed **solo or in pairs**. Both team members receive the same grade. All members are expected to contribute equally, and the report must include

a brief *Contribution Statement* specifying each member's responsibilities.

Academic Integrity

All code and written content must be your own. Use of AI-generated code without explicit disclosure is **strictly prohibited**. Plagiarism of any kind will result in a zero for the project and may be subject to further disciplinary action. You are encouraged to reference textbooks, documentation, and research papers — all sources must be cited properly.

3. Task 1 — Classification Algorithms

[10 Points pts]

3.1 Overview

Classification is one of the fundamental tasks in supervised machine learning. In this task, you will explore and compare a set of established classification algorithms, understand their theoretical foundations, and analyze their strengths and weaknesses.

3.2 Required Algorithms

You must study and describe the following six algorithms: (1) **k-Nearest Neighbors (k-NN)**, a distance-based, non-parametric classifier; (2) **Decision Trees**, which use tree-structured partitioning of the feature space; (3) **Random Forest**, an ensemble of decision trees via bagging; (4) **Support Vector Machines (SVM)**, a maximum-margin hyperplane classifier; (5) **Logistic Regression**, a probabilistic linear classifier; and (6) **Naive Bayes**, a generative Bayesian classifier.

3.3 Deliverables

Task 1 — Required Deliverables

1. **Theoretical Summary [3 pts]**: For each algorithm, write a concise description (150–250 words) covering its core mathematical idea or decision rule, the key hyperparameters and their effect on behavior, time and space complexity for both training and inference, and its best use cases along with known limitations.
2. **Comparison Table [3 pts]**: Produce a structured table comparing all six algorithms across the dimensions of interpretability, scalability, sensitivity to outliers, handling of high-dimensional data, and multi-class support.
3. **Algorithm Deep Dive [4 pts]**: Choose **one** algorithm and provide an in-depth analysis that includes a step-by-step mathematical derivation of the learning procedure, a hand-crafted example with a small dataset and manual calculations, a discussion of overfitting and underfitting behavior, and at least one diagram or figure illustrating the decision boundary or tree structure.

3.4 Useful Resources

The following references are recommended for this task. Bishop, C.M. (2006), *Pattern Recognition and Machine Learning*, Springer, provides rigorous mathematical treatment of all covered classifiers. Hastie, T., Tibshirani, R., & Friedman, J. (2009), *The Elements of Statistical Learning*, Springer, offers an excellent statistical perspective. The official Scikit-learn documentation at https://scikit-learn.org/stable/supervised_learning.html includes practical implementation guides and parameter references.

4. Task 2 — Python Implementation of Classification Models

[10 Points pts]

4.1 Overview

Theory becomes meaningful only when applied. In this task, you will implement, train, evaluate, and compare classification models using Python. Your code must be clean, well-documented, and reproducible.

4.2 Dataset Requirements

You must use **two datasets**. **Dataset A** must be a tabular dataset chosen from the UCI ML Repository (e.g., Iris, Wine, Breast Cancer, or Adult Income), with at least 500 samples and 5 or more features. **Dataset B** is your free choice: any publicly available dataset with at least 3 classes and at least 1,000 samples is acceptable.

4.3 Required Pipeline

Task 2 — Implementation Checklist

1. **Data Preprocessing [2 pts]**: Handle missing values, encode categorical features, and normalize or standardize numerical features. Split data into training, validation, and test sets using a 70/15/15 ratio. Document all preprocessing decisions and provide justification for each choice made.
2. **Model Training [3 pts]**: Train at least **four** classifiers from Task 1 on both datasets using `scikit-learn`. For each model, perform hyperparameter tuning using `GridSearchCV` or `RandomizedSearchCV` and report the best hyperparameters found.
3. **Evaluation [3 pts]**: Evaluate all models using accuracy, precision, recall, and F1-Score (both macro and weighted averages). Include a confusion matrix with visualization, a ROC curve with AUC score (one-vs-rest for multiclass problems), and 5-fold cross-validation results reported as mean $\pm$ standard deviation.
4. **Code Quality [2 pts]**: Code must be organized into modular functions or classes for each pipeline stage, include inline comments and docstrings throughout, and be accompanied by a `requirements.txt` listing all dependencies and a `README.md` with complete setup and run instructions.

4.4 Code Snippet Guidance

The following imports are recommended as a starting structure:


```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.pipeline import Pipeline
```

5. Task 3 — Dimensionality Reduction Techniques

[10 Points pts]

5.1 Overview

High-dimensional data is ubiquitous in machine learning. Dimensionality reduction is critical for visualization, noise removal, computational efficiency, and combating the curse of dimensionality.

5.2 Techniques to Cover

You must study and apply the following techniques: (1) **Principal Component Analysis (PCA)**, which performs linear projection via eigenvectors of the covariance matrix; (2) **t-SNE** (t-distributed Stochastic Neighbor Embedding), a non-linear method for 2D and 3D visualization; (3) **UMAP** (Uniform Manifold Approximation and Projection), a topology-preserving reduction technique; (4) **LDA** (Linear Discriminant Analysis), a supervised dimensionality reduction method. As an optional bonus, you may also explore **Autoencoders**, a neural network-based approach to unsupervised dimensionality reduction.

5.3 Deliverables

Task 3 — Required Deliverables

1. **Theoretical Background [3 pts]**: For PCA and t-SNE, provide the mathematical formulation of the objective function, an explanation of what structural information is preserved versus lost during projection, and a complexity analysis with scalability notes for large datasets.
2. **Practical Application [4 pts]**: Using the datasets from Task 2, apply PCA and plot the explained variance ratio curve (elbow plot), then visualize 2D projections using PCA, t-SNE, and UMAP with points colored by class label. Apply LDA and visualize the reduced space. Finally, retrain your best classifier from Task 2 on PCA-reduced data retaining 95% of variance and report the performance comparison.
3. **Analysis [3 pts]**: Write a 300–400 word discussion addressing which technique best preserved class separability, what trade-offs arose between dimensionality and classification accuracy, and when you would recommend each technique in a production setting.

Tip — PCA Variance Retention: When selecting the number of components for PCA, plot the cumulative explained variance ratio. A common heuristic is to retain components explaining 95% of variance. Formally, choose the smallest k such that:

$$\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i} \geq 0.95$$

where λ_i are the eigenvalues of the covariance matrix in descending order.

6. Task 4 — Deep Learning Fundamentals

[10 Points pts]

6.1 Overview

Deep learning has transformed modern AI. In this task, you will implement and analyze core deep learning concepts using [TensorFlow/Keras](#) or [PyTorch](#).

6.2 Core Concepts

You must demonstrate understanding and practical application of the following: feedforward neural networks (Multi-Layer Perceptrons), activation functions including ReLU, Sigmoid, Tanh, and Softmax, loss functions such as Cross-Entropy and Mean Squared Error, backpropagation and gradient descent variants (SGD, Adam, RMSProp), regularization techniques including Dropout, L1/L2 regularization, and Batch Normalization, and learning rate scheduling strategies.

6.3 Deliverables

Task 4 — Required Deliverables

1. **MLP Implementation [4 pts]**: Build a Multi-Layer Perceptron for classification on Dataset A from Task 2. Design an architecture with at least 3 hidden layers, experiment with at least two different activation functions, incorporate Dropout and Batch Normalization layers, and plot the training and validation loss and accuracy curves over all epochs.
2. **Optimizer Study [3 pts]**: Train the same architecture using SGD, Adam, and RMSProp. For each optimizer, report the convergence speed measured in epochs to plateau, plot all loss curves on the same axes for visual comparison, and provide a written discussion of your findings.
3. **Conceptual Questions [3 pts]**: Answer the following three questions in your report:
 - a) Explain the vanishing gradient problem and how modern architectures address it.
 - b) Why does Batch Normalization accelerate training? Derive or describe the normalization formula.
 - c) What is the role of the learning rate, and what happens when it is set too high or too low?

Key Formula — Cross-Entropy Loss:

For a C -class problem with true labels y and predicted probabilities $\hat{y}$:

$$\mathcal{L}_{CE} = - \sum_{c=1}^C y_c \log(\hat{y}_c)$$

The softmax function produces the probability distribution over classes:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

7. Task 5 — Convolutional Neural Networks

[10 Points pts]

7.1 Overview

Convolutional Neural Networks (CNNs) are the backbone of modern computer vision. In this task, you will build, train, and evaluate a CNN on an image classification task.

7.2 Dataset

Use **one** of the following image datasets:

Dataset	Description	Classes	Images
CIFAR-10	Natural images	10	60,000
MNIST	Handwritten digits	10	70,000
Fashion-MNIST	Clothing articles	10	70,000
SVHN	Street View house numbers	10	99,289

7.3 Deliverables

Task 5 — Required Deliverables

1. **CNN Architecture Design [3 pts]**: Build a custom CNN from scratch containing at least 3 convolutional blocks (each consisting of Conv + BatchNorm + ReLU + MaxPool), a classification head with at least 2 fully connected layers, and Dropout for regularization. Provide an architecture diagram clearly showing all layer dimensions and connections.
2. **Training & Evaluation [3 pts]**: Apply data augmentation including horizontal flip, random crop, and color jitter. Train for at least 30 epochs using a learning rate scheduler. Report test accuracy, per-class precision and recall, and include a confusion matrix. Plot training versus validation accuracy and loss over all epochs.
3. **Transfer Learning [2 pts]**: Fine-tune a pre-trained model such as ResNet-18, VGG-16, or MobileNet. Begin by freezing all layers except the final classification head, then unfreeze the entire model for full fine-tuning. Compare the accuracy and training time against your custom CNN from Deliverable 1.
4. **Visualization [2 pts]**: Visualize at least 8 learned filters from the first convolutional layer, display activation maps (feature maps) for 3 sample images using hooks or Grad-CAM, and show both correctly and incorrectly classified samples with their associated confidence scores.

CNN Key Operation — 2D Convolution:

Given input feature map I and kernel K of size $k \times k$, the output is:

$$(I * K)[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I[i + m, j + n] \cdot K[m, n]$$

The output spatial size after convolution with padding p and stride s :

$$\text{output size} = \left\lfloor \frac{W - k + 2p}{s} \right\rfloor + 1$$

where W is the input width.

8. Task 6 — General Overview & Integration

[10 Points pts]

8.1 Overview

This final task requires you to synthesize everything covered in Tasks 1–5 into a coherent, professional report and reflect on the broader landscape of deep learning.

8.2 Deliverables

Task 6 — Required Deliverables

1. **Deep Learning Algorithm Survey [4 pts]**: Write a structured survey of 600–800 words covering the following five architectures. For each, describe its purpose, key innovation, and a concrete real-world application: Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks, Generative Adversarial Networks (GANs), Transformers and the Attention Mechanism, Graph Neural Networks (GNNs), and Autoencoders together with Variational Autoencoders (VAEs).
2. **Comparative Analysis [3 pts]**: Produce a final comparison table or figure that integrates results across all tasks, including the best classical versus deep learning model performance per dataset, the impact of dimensionality reduction on each model, and a summary of training time, model size, and accuracy trade-offs observed throughout the project.
3. **Reflection & Conclusions [3 pts]**: Write a 400–500 word conclusion addressing the most valuable insight gained from the project, challenges encountered and how they were resolved, what you would do differently given more time or data, and the ethical considerations relevant to deploying the models you built.

9. Submission Guidelines

9.1 Required Files

Your submission must include the following, compressed in a single `.zip` archive named `ProjectGroup-[ID].zip`:

File / Folder	Description
<code>report.pdf</code>	Final written report (all 6 tasks, min 20 pages)
<code>notebooks/</code>	Jupyter notebooks for Tasks 2, 3, 4, and 5
<code>src/</code>	Python source files (<code>.py</code>) for modular code
<code>results/</code>	Saved model weights, figures, and evaluation logs
<code>README.md</code>	Setup instructions, dependencies, and how to run
<code>requirements.txt</code>	All Python package dependencies with versions

9.2 Report Format

The report must be a minimum of **20 pages** and a maximum of **40 pages**, excluding code listings and appendices. Use a standard academic font at 11–12pt size (Times New Roman, Arial, or the LaTeX default). All figures must be labeled, captioned, and referenced in the text; the same applies to all tables. Citations must follow either **IEEE** or **APA** format applied consistently throughout. For pairs, a **Contribution Statement** must appear before the references section.

9.3 Evaluation Rubric

Criterion	Description	Weight
Correctness	Solutions are technically sound and accurate	35%
Code Quality	Clean, documented, reproducible, modular code	20%
Report Quality	Clear writing, proper formatting, figures, citations	20%
Analysis	Depth of interpretation and critical thinking	15%
Creativity	Novel approaches, extra experiments, interesting findings	10%

10. Recommended Resources

10.1 Books

The following texts are strongly recommended. Goodfellow, I., Bengio, Y., & Courville, A. (2016), *Deep Learning*, MIT Press (<https://www.deeplearningbook.org>), is the standard reference for deep learning theory. Bishop, C.M. (2006), *Pattern Recognition and Machine Learning*, Springer, offers comprehensive coverage of probabilistic models and classical methods. Géron, A. (2022), *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, provides excellent practical guidance. Hastie, T., Tibshirani, R., & Friedman, J. (2009), *The Elements of Statistical Learning*, Springer, rounds out the theoretical foundation.

10.2 Online Courses & Tutorials

Several high-quality online resources are available at no cost. The **fast.ai** Practical Deep Learning for Coders course at <https://course.fast.ai> takes a top-down, application-first approach. The **DeepLearning.AI** specialization on Coursera provides structured instruction from Andrew Ng. **CS231n** (Stanford), Convolutional Neural Networks for Visual Recognition, is available at <http://cs231n.stanford.edu> and is particularly relevant to Task 5. **CS229** (Stanford), Machine Learning, available at <http://cs229.stanford.edu>, covers the theoretical foundations for Tasks 1–3.

10.3 Key Python Libraries

Library	Purpose	Documentation
scikit-learn	Classical ML	https://scikit-learn.org
TensorFlow/Keras	Deep learning	https://www.tensorflow.org
PyTorch	Deep learning	https://pytorch.org
pandas	Data manipulation	https://pandas.pydata.org
matplotlib / seaborn	Visualization	https://matplotlib.org
umap-learn	UMAP reduction	https://umap-learn.readthedocs.io
torchvision	Vision datasets	https://pytorch.org/vision

10.4 Datasets

The following repositories are recommended for finding suitable datasets. The UCI Machine Learning Repository at <https://archive.ics.uci.edu/ml/index.php> hosts hundreds of well-documented tabular datasets. Kaggle Datasets at <https://www.kaggle.com/datasets> offers a wide variety of real-world data. TensorFlow Datasets at <https://www.tensorflow.org/datasets> and HuggingFace Datasets at <https://huggingface.co/datasets> both provide ready-to-use datasets with standardized loading APIs.

11. Frequently Asked Questions

1. Can I use pre-built pipelines from AutoML tools?

No. All models must be manually configured and trained. Using AutoML platforms such as AutoSklearn or H2O is not permitted for the core tasks.

2. Are we allowed to use pre-trained weights for Task 5?

Yes, but only for the Transfer Learning sub-task (Task 5, Deliverable 3). Your custom CNN in Deliverable 1 must be trained from scratch.

3. What if a dataset is imbalanced?

Handle class imbalance explicitly. Document your chosen strategy (e.g., SMOTE, class weights, or undersampling) and justify it. Report both balanced and unweighted metrics so the reader can assess the impact.

4. Can we use GPU resources?

Yes. Google Colab (free tier) provides GPU access suitable for Tasks 4 and 5. You may also use Kaggle Kernels or any institutional HPC resources available to you.

5. How should pairs divide the work?

There is no enforced split. However, the Contribution Statement must clearly describe what each member did. Instructors may conduct brief individual oral checks if contributions appear uneven.

6. Can we use datasets not on the recommended list?

Yes, provided they meet the stated size and class requirements and are publicly available. Cite the dataset source in your report.

7. What citation format should we use?

Either IEEE or APA format is acceptable. Be consistent throughout the entire report — do not mix styles.

12. Grading Checklist

Use the checklist below to verify your submission before the deadline.

Report

- ☐ All 6 tasks are addressed with required sub-tasks completed.
- ☐ Report is at least 20 pages, well-structured with numbered sections.
- ☐ All figures are labeled, captioned, and referenced in text.
- ☐ All citations are included and bibliography follows IEEE or APA.
- ☐ Contribution Statement is included (for pairs).

Code

- ☐ Code runs end-to-end without errors.
- ☐ `requirements.txt` is present and complete.
- ☐ `README.md` explains how to set up and run the project.
- ☐ All notebooks have output cells cleared and re-run from scratch.
- ☐ Models are saved and loading from a checkpoint is demonstrated.

Results

- ☐ Evaluation metrics are reported for all models.
- ☐ Confusion matrices and ROC curves are included.
- ☐ Dimensionality reduction visualizations are included.
- ☐ CNN filter visualizations and Grad-CAM maps are included.
- ☐ Final comparison table integrating all tasks is included.

Submission

- ☐ Archive is named correctly: `ProjectGroup-[ID].zip`.
- ☐ Archive contains all required folders and files.
- ☐ Student name(s) and ID(s) appear on the report cover page.

**Good luck! We look forward to seeing
your innovative solutions and analyses.**

*School of Artificial Intel-
ligence and Data Science*